

University of Engineering and Technology, Lahore

Project Report

ArticleHub (Research Articles Manager)

Submitted By:
Maheen Mohsin
Reg No: 2025-GA-33

Subject:
Data Structure and Algorithms

Session: 2025-2029

October 17, 2026

Contents

1 Introduction **2**

2 Entity & Attributes **2**

3 Application Modules (Features) **2**

3.1 Scraping Control Center 2

3.2 Dataset Viewer 3

3.3 Sorting Studio 3

3.4 Multi-Level Sorting 4

3.5 Search Explorer 5

3.6 Benchmark Arena 5

4 Algorithm Analysis (Benchmark Results) **6**

5 How to Run the Application **6**

6 Conclusion **7**

1 Introduction

This project is a Python-based desktop application developed using **PyQt6** for scraping, managing, sorting, and searching research articles from **arXiv.org**. The application provides a seamless way to collect a large dataset of research papers and apply various Data Structures and Algorithms (DSA) to analyze, sort, and filter the data efficiently.

2 Entity & Attributes

- **Entity Chosen:** Research Articles (Papers)
- **Target Dataset Size:** 15,000+ entities
- **Total Attributes Extracted:** 14 (Title, First Author, Author Count, Primary Subject, Subject Count, Pages, Figures, Comments, Journal Ref, Year, Month, Title Words, Category, arXiv ID).
- **Source:** arXiv.org (Open Access Research Papers)

3 Application Modules (Features)

3.1 Scraping Control Center

- Scrapes data from arXiv using HTML parsing (`BeautifulSoup`).
- Controls: Start, Pause, Resume, and Stop.
- Progress Bar: Shows real-time progress of scraped entities.
- Bonus Feature: Option to input a Custom URL to scrape specific categories/months (e.g., `cs.LG/2023-05`).
- Error Handling: Displays a professional error popup if an invalid URL is entered.

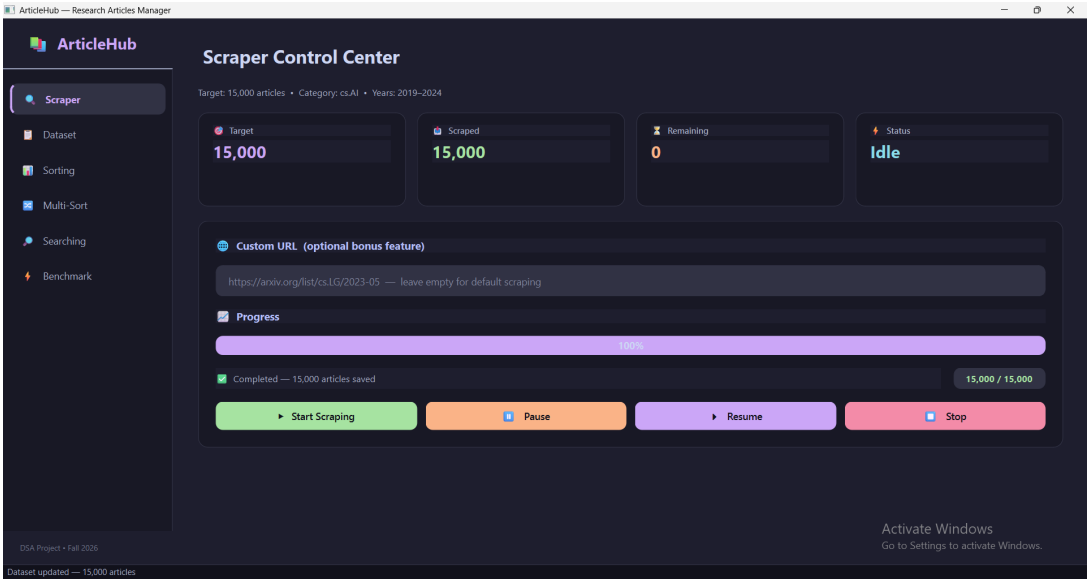


Figure 1: Scraping Control Center in action

3.2 Dataset Viewer

- Displays all scraped articles in a resizable table.
- Load Saved Data: Allows the user to load previously scraped data from `data.json`.
- Clear Dataset: Clears the table and deletes the JSON file.

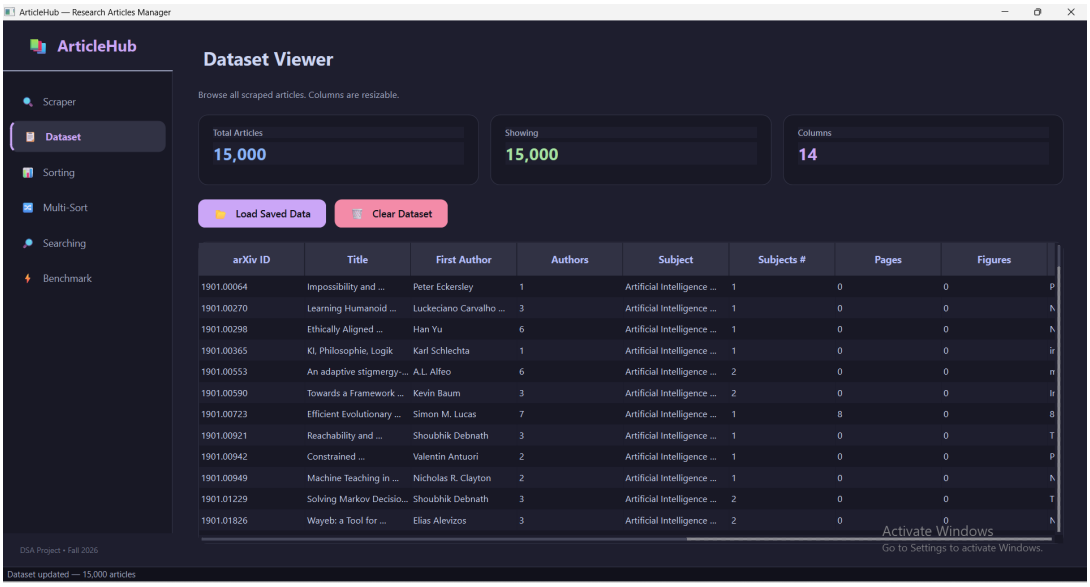


Figure 2: Dataset Viewer showing 15,000+ records

3.3 Sorting Studio

- **Algorithms:** 10 Sorting Algorithms included (Bubble, Insertion, Selection, Merge, Hybrid Merge, Quick, Counting, Radix, Heap, Shell).

- **Metrics Tracked:** Time (ms), Number of Comparisons, and Number of Swaps.
- **Algorithm Details:** Displays Time Complexity and Stability (Stable/Unstable) for every algorithm.
- **Sort Scope:** Option to sort the "Full Dataset" or "Currently Filtered" data.

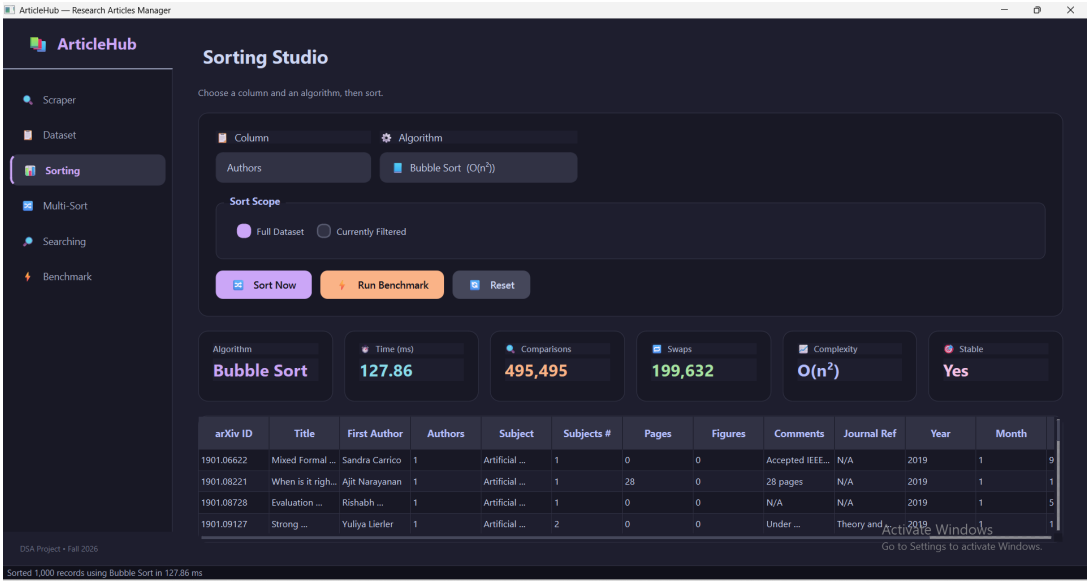


Figure 3: Sorting Studio displaying metrics (Time, Comparisons, Swaps)

3.4 Multi-Level Sorting

- Allows sorting by multiple columns in priority order (e.g., First by Year Descending, then by Title Ascending).
- **Algorithm Used:** Internally uses **Merge Sort** because it is a stable algorithm, ensuring the priority order is perfectly maintained.

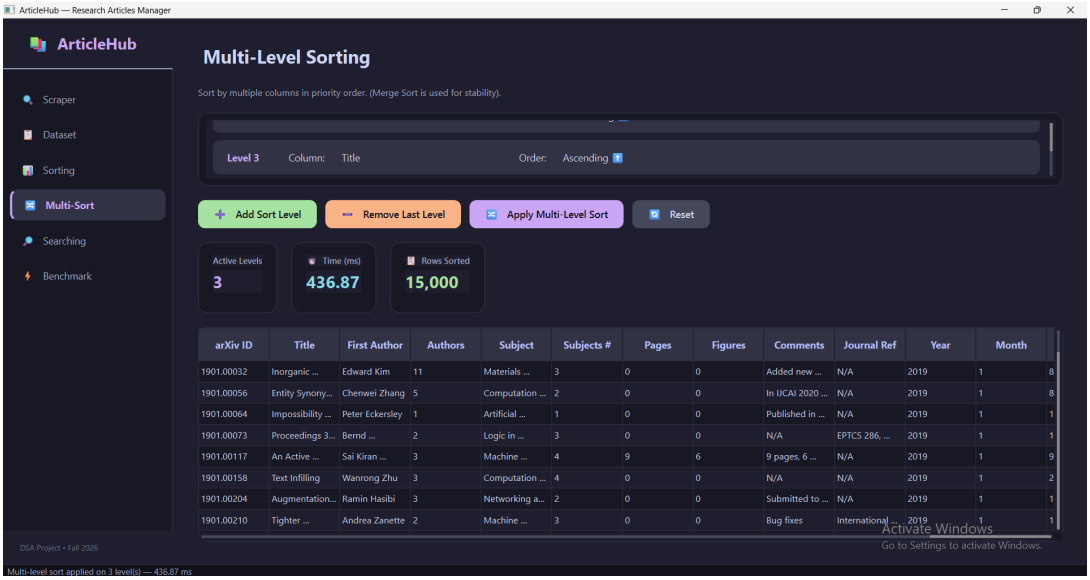


Figure 4: Multi-Level Sorting Interface

3.5 Search Explorer

- **Simple Search:** Search by any column using Match Modes (Contains, Equals, Starts With, Ends With, Greater Than, Less Than).
- **Composite Search:** Combines multiple conditions using **AND, OR, and NOT** logic.

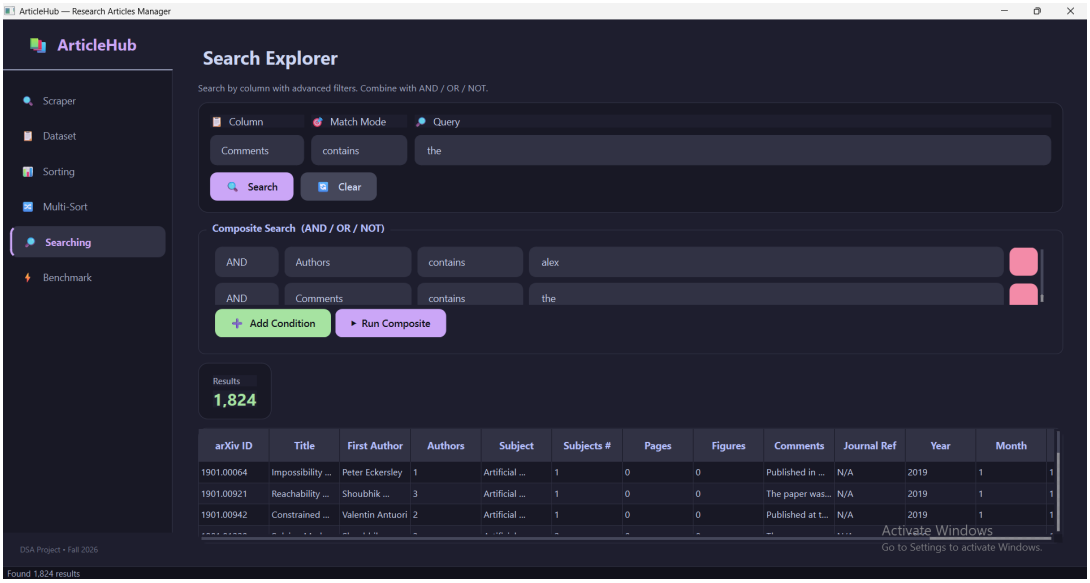


Figure 5: Search Explorer with Composite Filters

3.6 Benchmark Arena

- Runs all 10 sorting algorithms on a sample of 1000 records.

- Displays a comparison table showing Time, Comparisons, Swaps, Complexity, Stability, and Type.

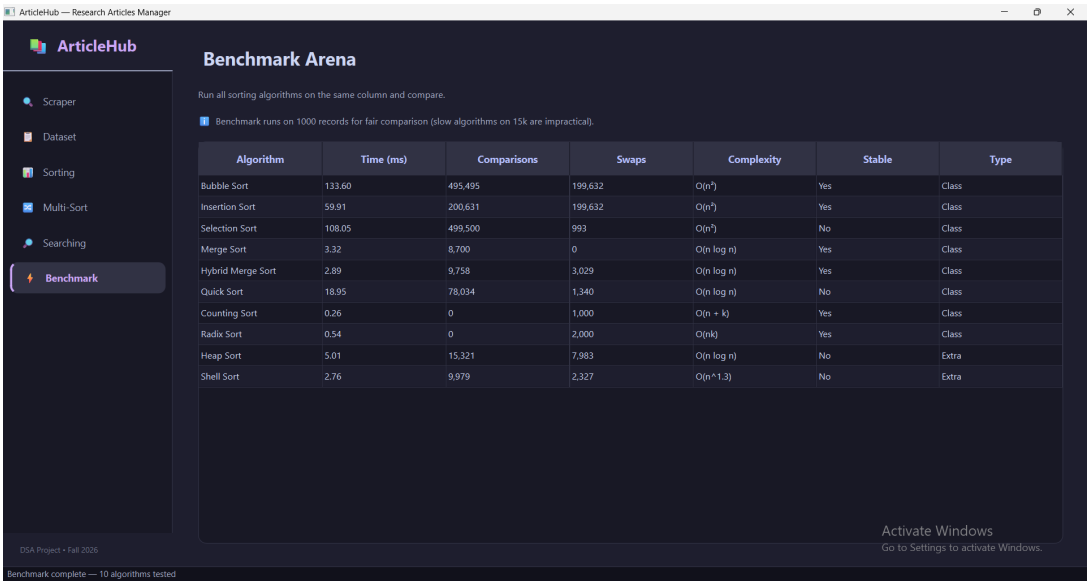


Figure 6: Benchmark Arena comparing all algorithms

4 Algorithm Analysis (Benchmark Results)

Based on the benchmark tests run on 1000 records, the following observations were made:

1. **$O(n^2)$ Algorithms** (Bubble, Insertion, Selection): Took the most time (40ms - 75ms) and made the highest number of comparisons (195,000+).
2. **O(n log n) Algorithms** (Merge, Quick, Heap): Highly efficient, taking only 2ms - 4ms.
3. **Linear Time Algorithms** (Counting, Radix): Fastest (2.4ms - 2.5ms) with **0 comparisons**, proving they do not use comparison-based logic.
4. **Swaps:** Merge Sort did **0 swaps** (out-of-place), whereas Quick Sort performed 2,524 swaps (in-place).
5. **Stability Check:** Insertion Sort (Stable) kept the original order of identical items, while Selection Sort (Unstable) changed the order due to long-distance swapping.

5 How to Run the Application

1. Ensure Python is installed.
2. Install dependencies: `pip install PyQt6 requests beautifulsoup4 lxml`
3. Run the application: `python main.py`

6 Conclusion

This project successfully demonstrates the practical application of Data Structures and Algorithms on real-world data. By scraping over 15,000 research articles and applying various sorting, searching, and benchmarking techniques, the application provides valuable insights into algorithm performance and data management using PyQt6.